

Demo Video Script & Talking Points

TAKE-HOME ASSIGNMENT

Banking CRM — Conversation-based Agentic AI · Ayush Rawat · 7–9 minutes

1. Demo Setup

1. Run: `npm run dev` → Frontend `http://localhost:5173` , API `http://localhost:3000/api/v1`

2. Login: `rm@bank.com` / `password123`

3. Optional: Swagger at `http://localhost:3000/api/docs`

2. Video Structure

Time	Section	Focus
0:00–0:45	Intro + problem	Use case understanding
0:45–2:00	Architecture	System design
2:00–6:30	3 live demos	Agent flow + output quality
6:30–8:00	Design decisions & trade-offs	Clarity
8:00–8:30	Wrap-up	Deliverables

3. Script

3.1 Intro (0:00–0:45)

SCREEN: LOGIN → DASHBOARD

“Hi, I’m Ayush. This is my take-home: a conversation-based Agentic AI for Banking CRM. A Relationship Manager asks in natural language to find high-value customers likely to convert for a personal loan and generate personalized WhatsApp messages. The system retrieves CRM data, scores conversion likelihood with explainable rules, recommends products, drafts WhatsApp messages, and requires human approval before outreach.”

3.2 Architecture (0:45–2:00)

SCREEN: README ARCHITECTURE / MONOREPO PACKAGES

“This is a monorepo: React + Vite frontend, NestJS backend, LangGraph agent workflow in agent-core, plus scoring-engine, recommendation-engine, prompts, and evaluation-engine packages. Data lives in PostgreSQL (1,000 seeded customers and related CRM data). Redis and Chroma are in the compose stack for caching and future semantic search. Flow: RM query → Planner → parallel retrieval tools → Scoring → Recommendation → Messaging → Audit → Response. Agents share AgentState. Tools query the database. The LLM is used for planning and message generation; conversion scores are deterministic and explainable.”

3.3 Demo 1 — Personal Loan (2:00–3:30)

SCREEN: AI AGENT TAB

Query: Find high-value customers likely to convert for a personal loan this month and generate personalized WhatsApp messages.

“Demo 1 is the core assignment use case.” [Run workflow] “The compact pipeline shows Plan → Data → Score → Match → Messages. Full step details open via View steps.” “Recommendations include conversion score, confidence, product, and reasons from business rules — income, credit, relationship, risk — not LLM-invented scores. Messages are personalized drafts in DRAFT status for human-in-the-loop approval.”

3.4 Demo 2 — Credit Card (3:30–4:45)

SCREEN: AI AGENT TAB

Query: Identify customers eligible for premium credit cards with strong transaction history.

“Demo 2 uses a different product intent. The Planner detects credit card; eligibility and ranking rules change. Same orchestration, different product path — the design is extensible, not hardcoded for one product. Results are ranked by confidence, then conversion score.”

3.5 Demo 3 — Approvals + CRM (4:45–6:30)

SCREEN: APPROVALS → CUSTOMERS → DASHBOARD / ANALYTICS

“Demo 3 covers RM operations. Messages appear under Approvals for Approve, Edit, and Reject — nothing is auto-sent.” “Customers supports search and CRM profiles (transactions, loans, notes) so the RM can verify agent output.” “Dashboard and Analytics show pending approvals, product mix, and evaluation benchmarks.”

Optional if time: Fixed deposit sample query, or Recommendations page.

3.6 Design Decisions & Trade-offs (6:30–8:00)

“Key decisions: 1. Deterministic scoring; LLM for language and planning. 2. Human approval before any outreach. 3. LangGraph for shared state, retries, and feature flags (memory, audit, streaming, cache). 4. Modular monorepo packages for scoring, recommendation, and prompts. Trade-offs: scoring is rule-based, not a trained ML model; WhatsApp is draft-only (no live send); Chroma is available for future semantic search; demo seed is 1,000 customers.”

3.7 Close (8:00–8:30)

“Deliverables: GitHub repository with modular agents, tools, and engines; README with architecture and setup; and this demo of three use cases plus trade-offs. Thank you.”

4. What Is Implemented (Talking Points)

System design

- Monorepo: `apps/backend` , `apps/frontend` , `packages ( agent-core , scoring-engine , recommendation-engine , prompts , evaluation-engine , shared-types , tools , config )`

- NestJS orchestrator + LangGraph workflow
- Prisma schema with banking + AI tables; seeded demo data
- Feature flags: memory, audit, streaming, cache

Agentic flow

- Natural language → Planner (intent + product type)
- Parallel retrieval: customers, transactions, CRM notes, loans, campaigns
- Score → eligibility → recommend → message → audit → response
- Shared `AgentState` across agents

Tools

Tool	Role
CustomerTool	High-value candidates
TransactionTool	Spend / balance signals
CrmTool	CRM notes
LoanTool	Loan history
CampaignTool	Prior campaigns
ScoringTool	Conversion score 0–100
RecommendationTool	Product fit + ranking
Message generation	Personalized WhatsApp drafts

Output quality

- Reasons and evidence on recommendations
- Confidence + conversion score
- Personalized draft messages
- Sorted by confidence, then score
- Human approval (Approve / Edit / Reject)

5. Three Use Cases to Show

1. **Personal loan + WhatsApp** — main assignment query
2. **Premium credit card** — different product path
3. **Approvals** — Approve / Edit / Reject

6. Key API Endpoints

Method	Endpoint	Purpose
POST	<code>/api/v1/auth/login</code>	JWT login
POST	<code>/api/v1/agent/query</code>	Run agent workflow

Method	Endpoint	Purpose
GET	/api/v1/messaging/pending	Pending approvals
PATCH	/api/v1/messaging/:id	Approve / reject / edit
GET	/api/v1/customers	Customer list
GET	/api/v1/analytics/summary	Dashboard metrics